

Module 7: Lesson 1

Reinforcement Learning: Monte Carlo Methods



Outline

- ▶ Reinforcement Learning: Wrapping up, what's next.
- ▶ Monte Carlo methods.
- ▶ Application: The windy gridworld.

Reinforcement Learning: Wrapping up, what's next

In this last Module, we will be able to build the skeleton of fully-fledged RL problems.

We are going to delve into three relevant RL algorithms: Monte Carlo, SARSA, and Q-learning, that exploit the markovian structure, the dynamic programming principles, and the simulation-based methods that we explored in previous Modules.

To gain some insights, our descriptions in Lessons 1 to 3 will be based on a windy gridworld setup that we developed in Module 5.

We will devote the Lesson 4 to the implementation of RL to derive the optimal daily trading choices of an investor on a particular stock.

Monte Carlo Methods

Let's consider again to the framework in Module 5 where an agent wants to make actions that maximize her cumulative discounted long-term rewards.

Monte Carlo methods represent an intuitive way to learn the state-action value function for a given policy.

In analogy with the policy iteration scheme we studied in Module 5, Monte Carlo methods first involve the computation of optimal state-action values for a given policy. We compute those values from repeated observations of the environment, and then simply average the returns observed after visits to each state.

Once we reach the end of an episode, the observed returns are used for policy evaluation, and then the policy is improved for all the states visited in the episode.

Monte Carlo methods: Visits and exploration

Within an episode, each occurrence of state s after taking action a is called a visit to (s, a) .

The pair (s, a) may be visited multiple times in the same episode.

“Every-visit” MC methods estimate the value of a state–action pair as the average of the rewards that have followed all the visits to it.

“First-visit” MC methods average the rewards only following the first time in each episode that the state–action pair was visited and the action was selected.

Many state–action pairs may never be visited by repeated experience. We again face the exploration–exploitation trade-off. To overcome this issue, we can opt for “exploring starts” and also ϵ -greedy policies, as we studied in Module 6.

Application: The windy gridworld

In the Jupyter Notebook to this Module, we implement the Monte Carlo Method to derive the optimal policies in a version of the gridworld, where gusts of wind hit the grid with varying intensity.

	Left	Right	Down
Up	Down	Down	Down
Left	Down	Down	Down
Right	Right	Right	

The figure displays the optimal policies that arise from RL that exploits Monte Carlo methods. The presence of wind tilts policies towards “down” movements.

Notice also that the optimal policy at cell 8 seems awkward. This is because it is a cell with a relatively low number of visits. Increasing the degree of exploration would change the result. Check it out yourself!

Summary of Lesson 1

In Lesson 1 we have looked at:

- ▶ The main insights underlying RL algorithms that exploit Monte Carlo.
- ▶ The application to computing the optimal choices in the windy gridworld.

⇒ **References for this Lesson:**

Sutton, Richard S., and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018. (see Chapter 5)

TO-DO NEXT: Now, please go to the associated Jupyter Notebook of this Lesson to verify the Monte Carlo implementation of the windy gridworld problem.

In the Next Lesson we will cover introduce the methods of Temporal Difference learning.